ALL

[redacted]

1) Problem in our previous code

* Memory inefficient
* Convergence → mini batch g.d

Mini batch G.D

32 320 32 32 ....

batch gradient descent

320 32 32 33

batch_size = 32
epoch = 25
n_sample = len(x_train_tensor)

start, end, step

for i in range(0, n_sample, batch_size)

x[i:i+batch_size]

for start_indx,

end_indx = start_indx,

X_train_tensor[start_indx:end_indx]

↳

Scalable System

dataset dataloader

1) No data shuffle

↳ shuffle = True

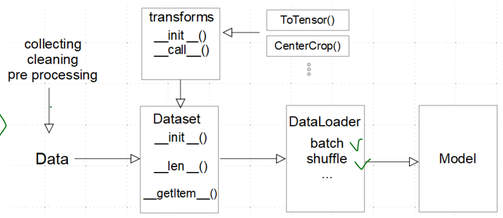
2) batching

abstracting

3) parallel

4)

Dataset & Dataloader



Row

data

transformation

CNN → block Resize

custom class

len

item → x, y

batch=2

Shuffle= True

Core abstraction

data

Dataset:

(7,8)

class MyDataset(Dataset):

init( )

len ⇒ length,

__getitem()

raw element

index

dataloader:

(7,8)

training

for pipeline

Collate fn

Data Transformation :

```python
class CustomDataset(Dataset):
  def __init__( self,features,labels):
    self.features = features
    self.labels = labels

  def __len__(self):
    return self.features.shape[0]

  def __getitem__(self,index):
    return self.features[index], self.labels[index]

```

transform

image :

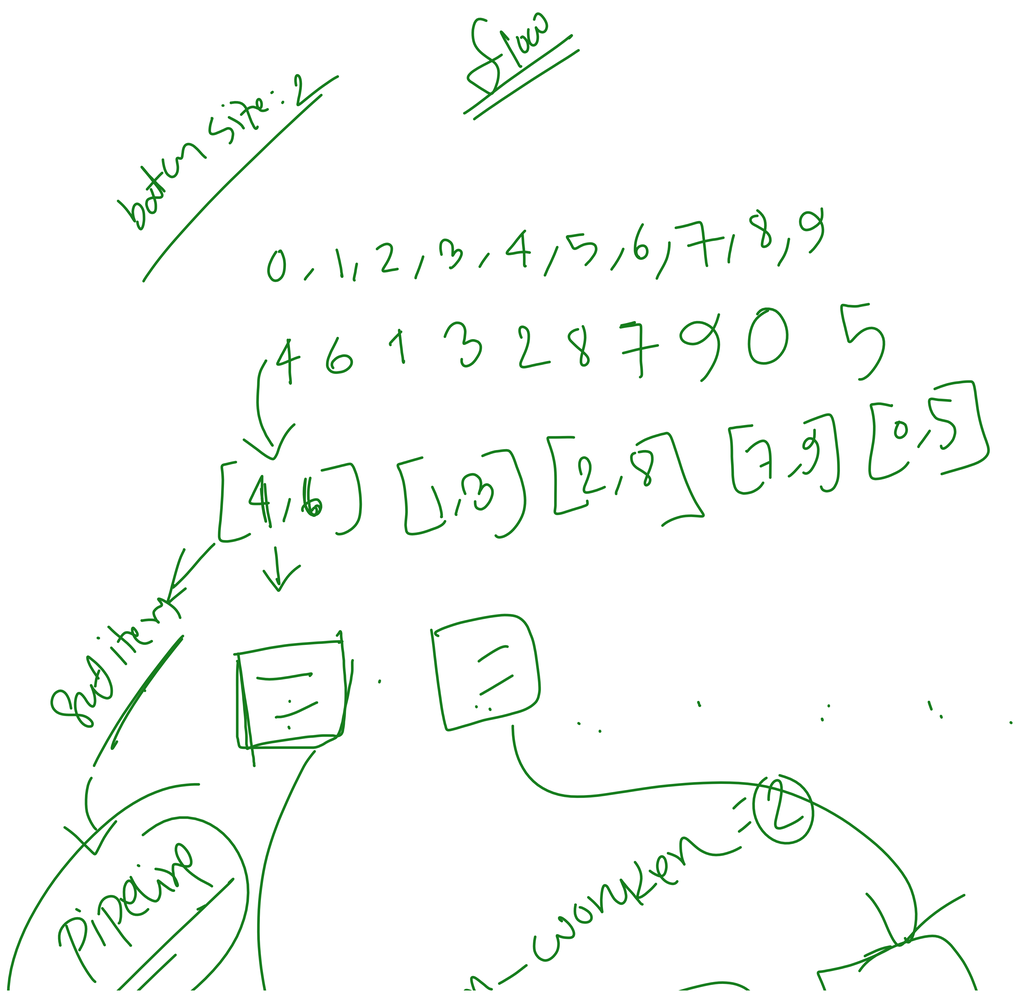
RGB

→ black & white

textual :

```python
import os
from PIL import Image
import torch
from torch.utils.data import Dataset
from torchvision import transforms

class CustomImageDataset(Dataset):

    def __init__(self, image_dir, labels, transform=None):
        self.image_dir = image_dir
        self.image_files = os.listdir(image_dir)
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        img_path = os.path.join(self.image_dir, self.image_files[idx])
        image = Image.open(img_path).convert("RGB")
        label = self.labels[idx]

        # Data transformation
        if self.transform:
```

```
        image = self.transform(image)

    return image, label


# Define CNN transformations
transform = transforms.Compose([
    transforms.Resize((128,128)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5,0.5,0.5], std=
[0.5,0.5,0.5])
])

# Create dataset
dataset = CustomImageDataset(
    image_dir="data/train",
    labels=[0,1,0,1],
    transform=transform
)
```



Sampler

→ sequential → 0→9 0,1,2,3 - - 9

→ random

→ custom sampler → imbalance

Collate fn : custom coll

Padding

It is used for 3 purposes generally:

i> To stack the data other than the first dimension.

ii> To stack data of different lengths.

iii> To put together data of different data types.

```python
from torch.utils.data import DataLoader
import torch

# simple dataset
dataset = [(2,0), (5,1), (7,0), (9,1)]

# custom collate function
def my_collate(batch):
    x = [item[0] for item in batch]
    y = [item[1] for item in batch]

    x = torch.tensor(x)
    y = torch.tensor(y)

    return x, y

loader = DataLoader(dataset, batch_size=2, collate_fn=my_collate)

for x, y in loader:
    print(x, y)
```

Sentence	Length
Hi there — - -	2
Let's go outside to play	5

- **`dataset` (Dataset)**: The dataset from which to load the data (map-style or iterable-style).
-
- **`batch_size` (int, optional)**: How many samples per batch to load (default: `1`).
-
- **`shuffle` (bool, optional)**: Set to `True` to shuffle the data at every epoch (default: `False`).
-
- **`sampler` (Sampler or Iterable, optional)**: Defines the strategy to draw samples from the dataset. Cannot be used with `shuffle=True`.

  `num_workers` (int, optional): How many subprocesses to use for data loading. `0` means data is loaded in the main process (default: `0`).
- **`collate_fn` (Callable, optional)**: Merges samples to form a mini-batch of Tensor(s).
- **`pin_memory` (bool, optional)**: If `True`, the data loader will copy Tensors into CUDA-pinned memory before returning them, allowing faster transfers to GPU.